



EECS 753 - Final project

Task scheduling on FreeRTOS

Presenter: Leo Cabezas



Motivation / Objectives

- **For the final project, I set myself to:**
 - Learn how to schedule tasks in a real-time operating system (RTOS).
 - Learn how to set up task scheduling experiments in an RTOS, and how to log and process task execution data.
 - Schedule a set of tasks both solo and concurrently to determine the effect of contentions on a shared resource.
 - Schedule a simple "attacker task" to further disrupt the behavior of other tasks and cause them to miss deadlines due to contention.

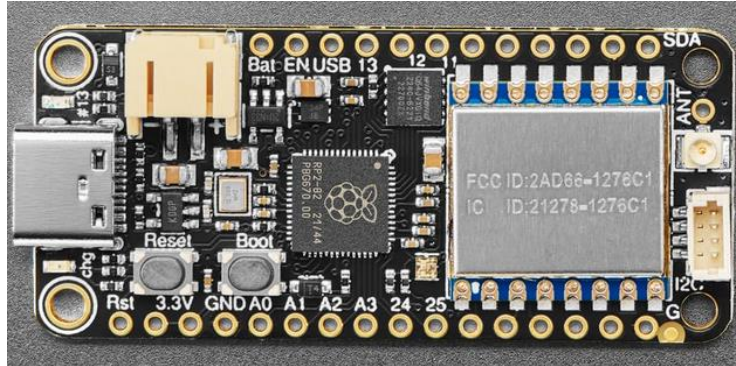
Motivation - What is FreeRTOS?



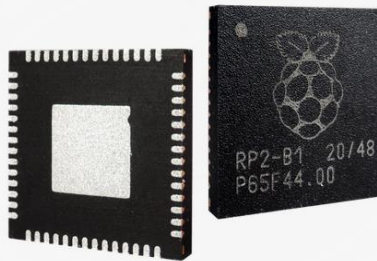
- FreeRTOS is a **free and open-source real-time operating system** developed by Amazon Web Services (AWS).
- It enables the programmer to **easily schedule real-time tasks on a microcontroller**, specifying task priority, assigned core, etc.
- It runs on many platforms, including:
 - SAMV7 (ARM Cortex-M7)
 - **SAMD20 (ARM Cortex-M0+)**
 - RISC-V RV32
 - Any many more!



Hardware - Target microcontroller



RP2040

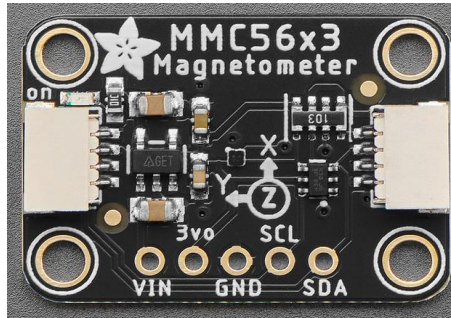


A microcontroller chip designed by
Raspberry Pi

- **Adafruit Feather RP2040 RFM95:** A microcontroller board including
 - The RP2040 microcontroller chip.
 - 8MB of on-board FLASH memory.
 - On-board RFM95 LoRa radio (unused here).

- **RP2040 microcontroller chip:** Manufactured by Raspberry Pi, this chip includes
 - 32-bit ARM Cortex M0+ dual core @ 133MHz
 - 264KB SRAM

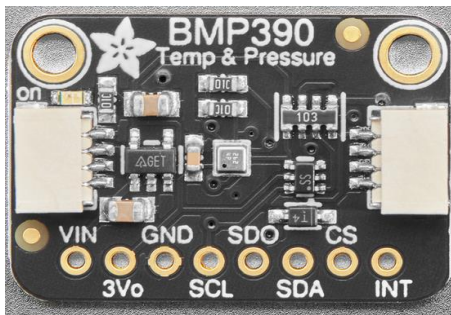
Hardware - External sensors



- **Adafruit MMC5603:** Three-axis magnetometer. Produces magnetic field strength data. Useful in "electronic compass" applications. Uses I²C.



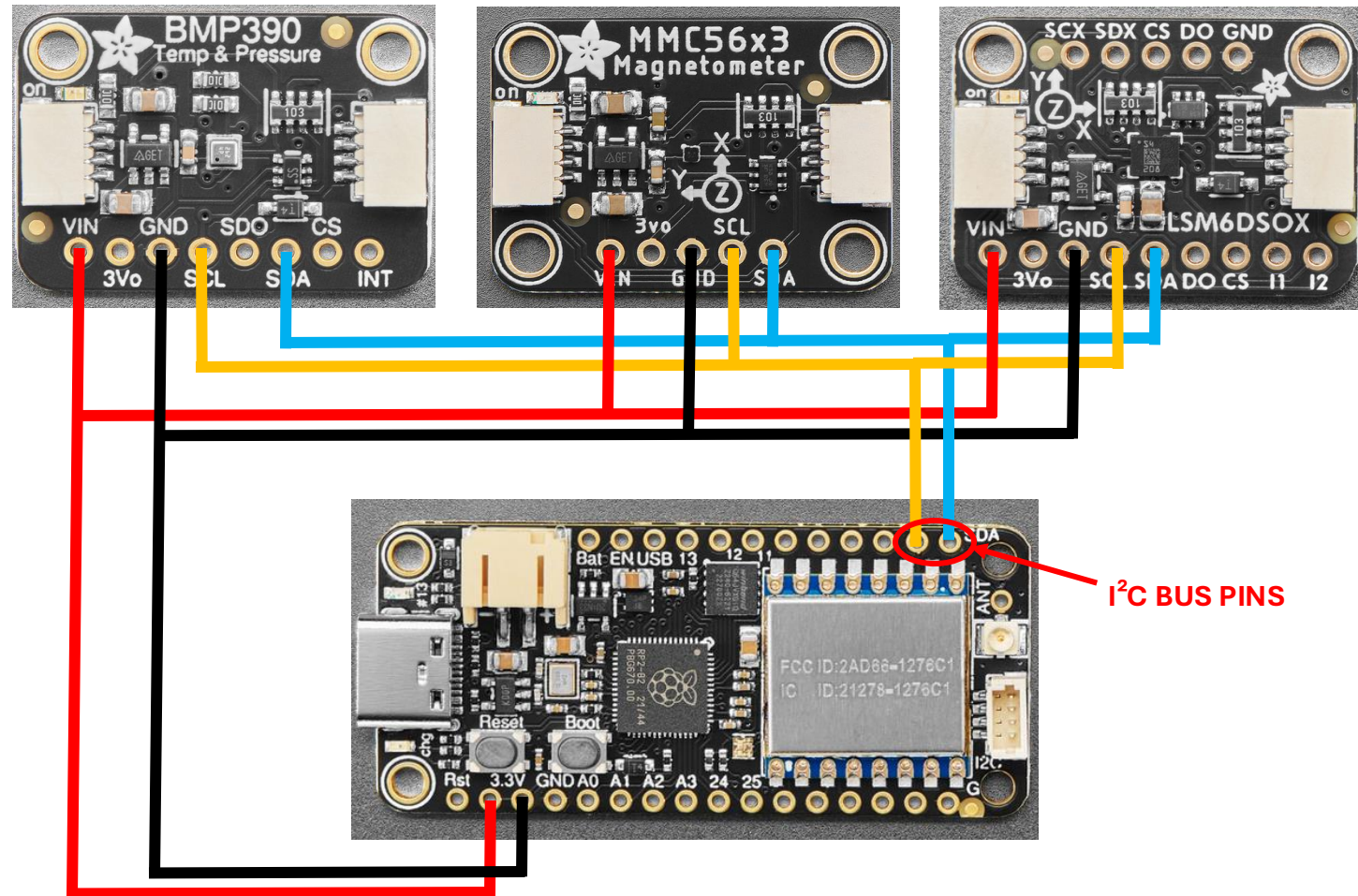
- **Adafruit LSM6DSOX:** A 2-in-1 accelerometer and gyroscope sensor. Produces linear and angular acceleration in all 3 axes. Uses the I²C protocol.



- **Adafruit BMP390:** Precision barometric pressure sensor and altimeter. Useful in applications that require device altitude. Uses the I²C protocol.



Hardware - Experiment setup (1)





Software - Magnetometer task (1)

**TASK
EXECUTION**

```
void vMagnetometer_UpdateHeading(void* pvParameters){
    (void)pvParameters;    // Parameters unused.

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(PERIOD_VMAGNETOMETER_UPDATEHEADING_MS);
    BaseType_t xWasDelayed;

    while (Earendil_Data->Experimental_Data.experiment_ongoing){
        TickType_t expected_wake_tick = xLastWakeTime + xFrequency;
        xWasDelayed = xTaskDelayUntil(&xLastWakeTime, xFrequency);
        TickType_t actual_wake_tick = xTaskGetTickCount();

        uint64_t start_task_us = time_us_64();

        {
            xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
            Earendil_Data->Magnetometer_Data.heading_deg = getHeading(hard_calibrated); // Task execution
            xSemaphoreGive(Earendil_Mutexes->i2c_mutex);

            uint32_t task_exec_time_us = (uint32_t)(time_us_64() - start_task_us);

            if (Earendil_Data->Experimental_Data.collect_magnetometer_logs){
                auto& log_array = Earendil_Data->Experimental_Data.magnetometer_logs;
                auto& log_array_idx = Earendil_Data->Experimental_Data.magnetometer_logs_idx;

                if (log_array_idx < Earendil_Experimental::DATA_LOG_SIZE){
                    log_array[log_array_idx].exec_time_us = task_exec_time_us;
                    log_array[log_array_idx].deadline_missed = (xWasDelayed == pdFALSE);
                    log_array[log_array_idx].expected_wake_tick = expected_wake_tick;
                    log_array[log_array_idx].actual_wake_tick = actual_wake_tick;
                    log_array[log_array_idx].wake_tick_difference = actual_wake_tick - expected_wake_tick;
                    log_array_idx++;
                }
            }
        }

        vTaskDelete(NULL);
    }
}
```

**DATA
LOGGING**

- **vMagnetometer_UpdateHeading - Task:**
- Calculates a heading to north based on the magnetometer sensor data, which is accessed through an I²C mutex.



Software - Magnetometer task (2)

```
void createTask_vMagnetometer_UpdateHeading(void){
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Magnetometer_Handles.task_vMagnetometer_UpdateHeading;
    xTaskCreate(
        vMagnetometer_UpdateHeading,           // Task function
        "vMagnetometer_UpdateHeading",         // Task name (for debugging)
        512,                                   // Task stack depth (in words, NOT bytes!)
        NULL,                                  // Task parameters at creation
        3,                                     // Real-time task priority
        task_handle_ptr                        // Task handle
    );
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<
        0                                     // Assigned core (options: 0, 1)
    );
}
```

- **vMagnetometer_UpdateHeading - Parameters:**
- Assigned core: 0
- Task priority: 3
- Period/Deadline: 40ms (25Hz)



Software - Accel. & Gyro. task (1)

**TASK
EXECUTION**

```
void vAccelGyro(void* pvParameters){
    (void) pvParameters;

    (void)pvParameters;    // Parameters unused.

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(PERIOD_VACCELGYRO_MS);
    BaseType_t xWasDelayed;

    while (Earendil_Data->Experimental_Data.experiment_ongoing){
        TickType_t expected_wake_tick = xLastWakeTime + xFrequency;
        xWasDelayed = xTaskDelayUntil(&xLastWakeTime, xFrequency);
        TickType_t actual_wake_tick = xTaskGetTickCount();

        uint64_t start_task_us = time_us_64();

        xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
        gyroReading();    // Task execution
        xSemaphoreGive(Earendil_Mutexes->i2c_mutex);

        uint32_t task_exec_time_us = (uint32_t)(time_us_64() - start_task_us);

        if (Earendil_Data->Experimental_Data.collect_altimeter_logs){
            auto& log_array = Earendil_Data->Experimental_Data.accelgyro_logs;
            auto& log_array_idx = Earendil_Data->Experimental_Data.accelgyro_logs_idx;

            if (log_array_idx < Earendil_Experimental::DATA_LOG_SIZE){
                log_array[log_array_idx].exec_time_us = task_exec_time_us;
                log_array[log_array_idx].deadline_missed = (xWasDelayed == pdFALSE);
                log_array[log_array_idx].expected_wake_tick = expected_wake_tick;
                log_array[log_array_idx].actual_wake_tick = actual_wake_tick;
                log_array[log_array_idx].wake_tick_difference = actual_wake_tick - expected_wake_tick;
                log_array_idx++;
            }
        }
    }

    vTaskDelete(NULL);
}
```

**DATA
LOGGING**

- **vAccelGyro - Task:**
- Produces linear/angular acceleration readings based on the gyroscope and accelerometer sensor data, which is accessed via an I²C mutex.

Software - Accel. & Gyro. task (2)



```
// =====  
// vAccelGyro  
// =====  
  
void createTask_vAccelGyro(void){  
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->AccelGyro_Handles.task_vAccelGyro;  
    xTaskCreate(  
        vAccelGyro,                // Task function  
        "vAccelGyro",             // Task name (for debugging)  
        512,                      // Task stack depth (in words, NOT bytes!)  
        NULL,                    // Task parameters at creation  
        2,                       // Real-time task priority  
        task_handle_ptr           // Task handle  
    );  
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<  
        0                        // Assigned core (options: 0, 1)  
    );  
}
```

- **vAccelGyro - Parameters:**
- Assigned core: 0
- Task priority: 2
- Period/Deadline: 80ms (12.5Hz)

Software - Altimeter task (1)



```
void vAltimeter(void* pvParameters){
    (void)pvParameters;    // Parameters unused.

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(PERIOD_VALTIMETER_MS);
    BaseType_t xWasDelayed;

    while (Earendil_Data->Experimental_Data.experiment_ongoing){
        TickType_t expected_wake_tick = xLastWakeTime + xFrequency;
        xWasDelayed = xTaskDelayUntil(&xLastWakeTime, xFrequency);
        TickType_t actual_wake_tick = xTaskGetTickCount();

        uint64_t start_task_us = time_us_64();

        {
            xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
            altimeterRead();                // Task execution
            xSemaphoreGive(Earendil_Mutexes->i2c_mutex);

            uint32_t task_exec_time_us = (uint32_t)(time_us_64() - start_task_us);
        }

        if (Earendil_Data->Experimental_Data.collect_altimeter_logs){
            auto& log_array = Earendil_Data->Experimental_Data.altimeter_logs;
            auto& log_array_idx = Earendil_Data->Experimental_Data.altimeter_logs_idx;

            if (log_array_idx < Earendil_Experimental::DATA_LOG_SIZE){
                log_array[log_array_idx].exec_time_us = task_exec_time_us;
                log_array[log_array_idx].deadline_missed = (xWasDelayed == pdFALSE);
                log_array[log_array_idx].expected_wake_tick = expected_wake_tick;
                log_array[log_array_idx].actual_wake_tick = actual_wake_tick;
                log_array[log_array_idx].wake_tick_difference = actual_wake_tick - expected_wake_tick;
                log_array_idx++;
            }
        }
    }

    vTaskDelete(NULL);
}
```

**TASK
EXECUTION**

**DATA
LOGGING**

- **vAltimeter - Task:**
- Generates altimetry and pressure data, accessed via an I²C mutex.



Software - Altimeter task (2)

```
// =====  
// vAltimeter  
// =====  
  
void createTask_vAltimeter(void){  
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Altimeter_Handles.task_vAltimeter;  
    xTaskCreate(  
        vAltimeter,                    // Task function  
        "vAltimeter",                 // Task name (for debugging)  
        512,                          // Task stack depth (in words, NOT bytes!)  
        NULL,                        // Task parameters at creation  
        1,                          // Real-time task priority  
        task_handle_ptr              // Task handle  
    );  
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<  
        0                            // Assigned core (options: 0, 1)  
    );  
}
```

- **vAltimeter - Parameters:**
- Assigned core: 0
- Task priority: 1
- Period/Deadline: 120ms (~8.33Hz)

Software - Supervisor task (1)



```
void vExperimentSupervisor(void* pvParameters){
    (void)pvParameters;

    Earendil_Data->Experimental_Data.collect_magnetometer_logs = false; // Enable Earendil_Magnetometer logs.
    Earendil_Data->Experimental_Data.collect_accelgyro_logs      = false; // Enable Earendil_AccelGyro logs.
    Earendil_Data->Experimental_Data.collect_altimeter_logs     = false; // Enable Earendil_Altimeter logs.

    Earendil_Data->Experimental_Data.experiment_ongoing = true; // Start experiment.
    vTaskDelay(pdMS_TO_TICKS(EXPERIMENT_DURATION_MS)); // Wait for experiment to conclude.

    Earendil_Data->Experimental_Data.experiment_ongoing = false; // Stop experiment.
    vTaskDelay(pdMS_TO_TICKS(DELAY_AFTER_EXPERIMENT_MS)); // Wait a bit for all tasks to stop.

    if (Earendil_Data->Experimental_Data.collect_magnetometer_logs){
        dumpLogs(
            "magnetometer",
            Earendil_Data->Experimental_Data.magnetometer_logs,
            Earendil_Data->Experimental_Data.magnetometer_logs_idx
        ); // Dump Earendil_Magnetometer logs to USB serial via printf.
    }
    if (Earendil_Data->Experimental_Data.collect_accelgyro_logs){
        dumpLogs(
            "accelgyro",
            Earendil_Data->Experimental_Data.accelgyro_logs,
            Earendil_Data->Experimental_Data.accelgyro_logs_idx
        ); // Dump Earendil_AccelGyro logs to USB serial via printf.
    }
    if (Earendil_Data->Experimental_Data.collect_altimeter_logs){
        dumpLogs(
            "altimeter",
            Earendil_Data->Experimental_Data.altimeter_logs,
            Earendil_Data->Experimental_Data.altimeter_logs_idx
        ); // Dump Earendil_Altimeter logs to USB serial via printf.
    }

    vTaskDelete(NULL);
}
```

**DATA
OUTPUT
TO USB
SERIAL**

- **vExperimentSupervisor - Task:**
- Limits the duration of the experiment (here 30000 ms) by toggling a control flag off. Tasks only run while this flag is enabled.
- Outputs data logs for all modules to USB serial in CSV format after the time is up.
- **Data logs for each iteration include:**
 - Execution time
 - Whether a deadline was missed



Software - Supervisor task (2)

```
// =====  
// vExperimentSupervisor  
// =====  
  
void createTask_vExperimentSupervisor(){  
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Experimental_Handles.task_vExperimentSupervisor;  
    xTaskCreate(  
        vExperimentSupervisor,           // Task function  
        "vExperimentSupervisor",        // Task name (for debugging)  
        512,                             // Task stack depth (in words, NOT bytes!)  
        NULL,                           // Task parameters at creation  
        30,                             // Real-time task priority  
        task_handle_ptr                 // Task handle  
    );  
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<  
        0                               // Assigned core (options: 0, 1)  
    );  
}
```

- **vExperimentSupervisor - Parameters:**
- **Assigned core:** 0
- **Task priority:** 30 (higher than any other task)
- **Period/Deadline:** Not applicable (only runs once)



Software - Task summary

	Supervisor task	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (ms)	N/A	40	80	120
Assigned core	0	0	0	0
Priority	30	3	2	1



Results - Running tasks solo

	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (us)	40000	80000	120000
Average execution time (us)	13257	1749	70
Max. exec. time (us)	13493	2421	414
Execution time standard deviation (us)	236	46	30
Deadlines missed	0	0	0

- If we approximate WCETs by max. exec. times, we get a total utilization of:
- $U \approx 0.371 < \ln(2)$ (SCHEDULABLE)
- This calculation, however, fails to account for shared resource contention (I²C bus).



Results - Running tasks concurrently

	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (us)	40000	80000	120000
Average execution time (us)	13093	8064	14265
Max. exec. time (us)	13372	14352	14474
Execution time standard deviation (us)	338	6153	33
Deadlines missed	0	0	0

- The effects of I²C bus contention are clear. Lower priority tasks are slowed down significantly compared to 'solo'. However, the **highest priority task is not slowed down. This demonstrates that PIP is being applied to FreeRTOS mutexes effectively.**
- $U \approx 0.634 < \ln(2) \implies$ **STILL SCHEDULABLE**

Attacks - I²C bus hog (1)



```
void vI2CBusHog(void* pvParameters){
    (void)pvParameters;

    volatile uint32_t x = 0;

    while (1){
        xSemaphoreTake(Earendil_Mutexes->i2c_mutex, portMAX_DELAY);
        // Hold the mutex for a while
        for (int i = 0; i < 400000; i++){
            x += i;
        }
        xSemaphoreGive(Earendil_Mutexes->i2c_mutex);
    }
}
```



Attacks - I²C bus hog (2)

```
// =====  
// vI2CBusHog  
// =====  
  
void createTask_vI2CBusHog(){  
    TaskHandle_t* task_handle_ptr = &Earendil_Handles->Experimental_Handles.task_vI2CBusHog;  
    xTaskCreate(  
        vI2CBusHog,                // Task function  
        "vI2CBusHog",             // Task name (for debugging)  
        512,                       // Task stack depth (in words, NOT bytes!)  
        NULL,                     // Task parameters at creation  
        0,                        // Real-time task priority  
        task_handle_ptr           // Task handle  
    );  
    vTaskCoreAffinitySet(*task_handle_ptr, 1 <<  
        1                          // Assigned core (options: 0, 1)  
    );  
}
```

- **vI2CBusHog- Parameters:**
- **Assigned core:** 1
- **Task priority:** 0
- **Period/Deadline:** Not applicable



Results - Effects of the I²C bus hog attack

	Magnetometer task	Accel&Gyro task	Altimeter task
T := Period/Deadline (us)	40000	80000	120000
Average execution time (us)	28734	17260	17130
Max. exec. time (us)	46084	27704	27661
Execution time standard deviation (us)	10490	5079	5862
Deadlines missed	138	0	0

- Even though the attacker has the least priority and is scheduled on a different core, it **accessing and overloading a shared resource (the I²C bus) results in higher WCETs for all tasks, and deadline misses** for the task with the smallest deadline (magnetometer task).
- $U \approx 1.729 > 1 \implies$ **NOT SCHEDULABLE**

Attacks - Memory thrasher



```
void vMemThrash(void* pvParameters){
    (void)pvParameters;
    constexpr uint32_t size = 30050;
    static volatile uint32_t buffer[size];

    while (1){
        for (int i = 0; i < size; i++){
            buffer[i] += i;
        }

        for (int i = size - 1; i >= 0; i--){
            buffer[i] ^= i;
        }
    }
}
```

- A different attempt at an "attacker task", which was unsuccessful in causing significant delays or producing missed deadlines (hence why the data table is omitted, to avoid redundancy).

End - Brief Q&A

